Reinforcement Learning

ELG 5218 – Value-Based, Policy Gradient, and Actor-Critic Methods

Bingze Xia

University of Ottawa

April 1, 2026

Outline: Chapter 35 – Reinforcement Learning

1. **RL Overview**
   - Don't know the rules, have to try, explore and learn.
2. **Value-Based RL**
   - TD learning, SARSA, Q-learning, DQN
3. **Policy-Based RL**
   - Policy gradient, Actor-Critic, PPO, DDPG
4. **Exploration & Practical Issues**
   - off-policy, SAC
5. **Summary & Connections**

Part 1: RL Overview

From known models to learning from interaction

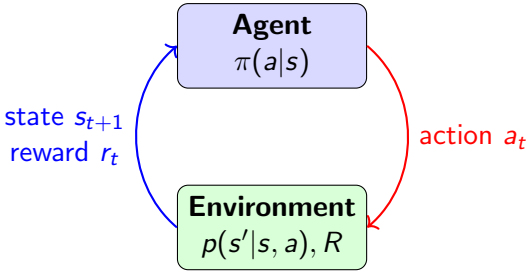
Planning vs. Learning

Ch34: Planning - You have the perfect map

Model $p(s'|s,a)$ and $R(s,a)$ are **known**.
$\rightarrow$ Value Iteration, Policy Iteration
$\rightarrow$ Compute π^* without interacting with environment

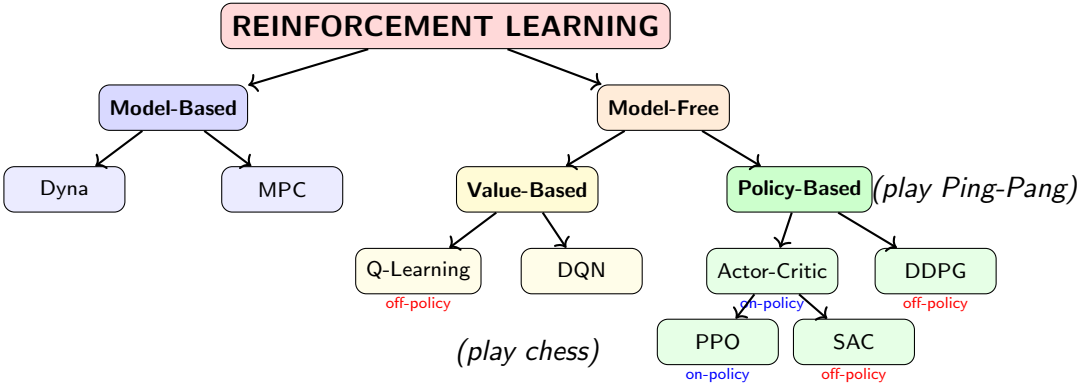
Ch35: Reinforcement Learning

Model is **unknown**.
$\rightarrow$ Agent must **interact** with environment
$\rightarrow$ Learn from observed (s,a,r,s') tuples



The agent–environment interaction loop. At each step: observe s_t, take a_t, receive r_t, transition to s_{t+1}.

RL Taxonomy



Model-based: Plan within a learned model (a "guessed" world). Pro: Data-efficient. Con: Sensitive to model inaccuracies. **Model-free:** Learn directly via trial-and-error interactions with the environment.

Summary of Popular RL Methods (Murphy Table 35.1)

Method	What it learns	On/Off-policy	Section
SARSA	$Q(s, a)$	On	35.2.4
Q-learning	$Q(s, a)$	Off	35.2.5
REINFORCE	$\pi(a\|s)$	On	35.3.2
A2C/A3C	$\pi(a\|s)$, $V(s)$	On	35.3.3
TRPO/PPO	$\pi(a\|s)$, $A(s, a)$	On	35.3.4
DDPG	$a = \mu(s)$, $Q(s, a)$	Off	35.3.5
SAC	$\pi(a\|s)$, $Q(s, a)$	Off	35.6.1
Model-based	$p(s'\|s, a)$	Off	35.4

Key distinctions:

- **On-policy:** data must come from the *current* policy (try a restaurant yourself)
- **Off-policy:** can take experience from *any* policy (read comments)

On-Policy vs. Off-Policy

On-Policy

Learn from data generated by **current** policy π.

- Simpler, more stable
- Must discard data when π changes
- Examples: SARSA, REINFORCE, PPO

Off-Policy

Learn from data generated by **any** policy π_b.

- Can reuse old data (replay buffer)
- More sample efficient
- Can be unstable (deadly triad)
- Examples: Q-learning, DQN, SAC

Off-policy is essential when real-world interaction is expensive (e.g., robotics, clinical trials, UAV systems).

Part 2: Value-Based RL

Learn Q^*, then derive π^*

Monte Carlo (MC) RL

Idea: Play a full game (an episode), sum up the actual rewards, and average them over many games to build your value table.

Algorithm:

1. Sample a full trajectory $\tau = (s_0, a_0, r_0, s_1, \ldots, s_T)$ using policy π
2. Compute return: $G_t = r_t + \gamma r_{t+1} + \cdots + \gamma^{T-t-1} r_{T-1}$
3. Update: $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \eta \left(G_t - Q(s_t, a_t)\right)$

Properties:

+ **Unbiased** estimate of Q^π
+ Does not require model knowledge
− Must wait until episode ends (no incremental updates)
− **High variance** (sum of many random rewards)

Temporal Difference (TD) Learning

Key insight: Don't wait for the full return. Instead, **bootstrap**: use current estimate $V(s_{t+1})$ to approximate future return.

TD(0) Update Rule: using learning rate Eta

$$V(s_t) \leftarrow V(s_t) + \eta \underbrace{[r_t + \gamma V(s_{t+1}) - V(s_t)]}_{\text{TD error } \delta_t}$$

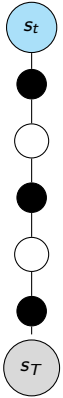
Analogy: Driving to Toronto. Update your total ETA once you reach Kingston

Properties:

- \+ Updates after **every step** (no need to wait for episode end)
- \+ **Lower variance** than MC
- − **Biased** (Since you use your own estimate)

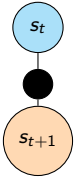
MC vs. TD vs. DP: Backup Diagrams (Murphy Fig. 35.2)

Monte Carlo



Uses full return G_t
(no bootstrap)

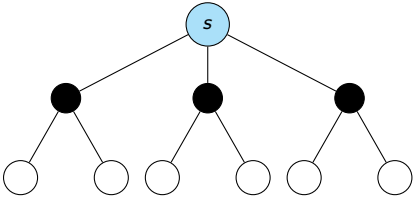
Temporal Difference



Uses $r_t + \gamma V(s_{t+1})$
(1-step bootstrap)

A sample from DP

Dynamic Programming



Full model: $\max_a \sum_{s'} p(s'|s,a)$
(requires p)

last section's "God's view":
All possible parallel futures

SARSA: On-Policy TD Control

Idea: Use TD to learn Q^π while following and improving π.

SARSA Update (uses s, a, r, s', a')

$$Q(s,a) \leftarrow Q(s,a) + \eta\left[r + \gamma Q(s',a') - Q(s,a)\right]$$

where $a' \sim \pi(s')$ is the action the agent **actually takes** next.

Name: comes from the tuple (S, A, R, S', A').

Properties (Slip on Ice):

- **On-policy:** Evaluates the exact path you are walking, including clumsy mistakes.
- **GLIE ($\epsilon \to 0$):** Like "growing up." If you gradually stop taking random risky steps, you eventually find the perfect route (Q^*).
- **Fixed ϵ-greedy:** Like a "cautious survivor." a sub-optimal path keeps you away from cliffs.

Q-Learning: Off-Policy TD Control

Key modification: Replace sampled a' with greedy max.

Q-Learning Update (Watkins, 1992) b: all possible actions in s'

$$Q(s,a) \leftarrow Q(s,a) + \eta \left[r + \gamma \max_b Q(s',b) - Q(s,a) \right]$$

Why this matters (The "Blind Optimist"):

- **Off-policy:** Always assumes a perfect future (max), even if you're actually slipping on ice.
- **Data Re-use (Reading Reviews):** Doesn't need to walk the path itself. Can learn the perfect strategy just by watching others.
- **The Risk-Taker (Cliff Edge):** Finds the theoretical fastest route, but will confidently walk you right off a cliff if you still have an ϵ chance to slip.

TD error δ: a sample-based approximation of the **Bellman error**.

SARSA vs. Q-Learning: Side by Side

	SARSA	**Q-Learning**
Update target	$r + \gamma Q(s', a')$	$r + \gamma \max_b Q(s', b)$
Next action	$a' \sim \pi(s')$	$\text{argmax}_b\, Q(s', b)$
On/Off-policy	**On**-policy	**Off**-policy
Learns	Q^π	Q^*
Behavior	Cautious (respects π)	Aggressive (assumes best)

Example: In a cliff-walking problem:

- SARSA (on-policy) learns a **safe** path away from the cliff edge (because ϵ-greedy occasionally falls off)
- Q-learning (off-policy) learns the **optimal** path along the cliff edge

Q-Learning Example: 1D Grid World (Murphy Fig. 35.4)

Ep.	Q at start Up Down	**Transition** (s, a, r, s')	**Target** $r + \gamma Q^*(s', a^*)$	Q **updated**
1	all 0	$(S_3, \downarrow, 1, S_{T2})$	1	$Q(S_3,\downarrow) = 1$
2	$Q(S_3,\downarrow) = 1$	$(S_2, \downarrow, 0, S_3)$ $(S_3, \downarrow, 1, S_{T2})$	$0 + 0.9 \times 1 = 0.9$ 1	$Q(S_2,\downarrow) = 0.9$ $Q(S_3,\downarrow) = 1$
3	$Q(S_2,\downarrow) = 0.9$	$(S_1, \downarrow, 0, S_2)$ $\dots$	$0 + 0.9 \times 0.9 = 0.81$ $\dots$	$Q(S_1,\downarrow) = 0.81$

Note: Q-Learning is a *sampled* Bellman update. **Value Iteration** uses a known full model; Q-learning learns step-by-step from real experience. Both eventually converge to the same Q^*.

Q-Learning Example: 1D Grid World - Matlab example

Converged Q^* values (same as Ch34, $\gamma = 0.9$):

	Up	Down
S_1	0	0.81
S_2	0.73	0.9
S_3	0.81	1.0

Key Takeaway: Q-Learning learns the same Q^* without knowing the model ("the map").
The Trade-off: It requires significantly more real-world experience (exploration).

Double Q-Learning: Fixing Maximization Bias

The Problem: Maximization Bias (Over-optimism)

- **Analogy:** You ask 10 friends for restaurant reviews and always pick the one with the highest score.
- **The Trap:** A restaurant might not be good; it just got "lucky" with high rating. By picking the *max*, you are biased toward lucky outliers.
- **In Q-Learning:** If Q accidentally overestimates an action, we pick it and update it $\rightarrow$ a dangerous positive feedback loop!

The Solution: Two-Person Verification (Decoupling)

$$Q_1(s,a) \leftarrow Q_1(s,a) + \eta\big[r + \gamma Q_2\big(s', \text{argmax}_{a'} Q_1(s',a')\big) - Q_1(s,a)\big]$$

- **Idea:** Use Q_1 to **nominate** (pick the action), and Q_2 to **verify** (estimate the value). This breaks the over-optimistic cycle.
- **Legacy:** This is the ancestor of *Target Networks* in DQN and *Twin Critics* in TD3. Remember this trick!

Deep Q-Network (DQN) – Mnih et al., 2015

Problem: Table is too small, not valid for infinite s-a pairs
Solution: Replace the table with a Brain (a Neural Network)

Two key innovations for stability:

1. Experience Replay

- Store transitions (s, a, r, s') in buffer $\mathcal{D}$
- Sample mini-batches uniformly from $\mathcal{D}$
- Breaks temporal correlation
- Reuses data $\rightarrow$ more efficient

2. Target Network

- Use a "frozen" twin to set the goal.
- Don't update the target at every step.
- **Stability:** Only sync with the main brain periodically.

DQN Improvements

DQN Loss: (Minimizing the distance between our "guess" and the "target" we set)

$$\mathcal{L}^{\text{DQN}}(\boldsymbol{w}) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(r + \gamma \max_{a'} Q_{\boldsymbol{w}^-}(s', a') - Q_{\boldsymbol{w}}(s, a)\right)^2\right]$$

Three Key Upgrades After DQN:

1. **Double DQN (Fix Optimism):** Uses the same "two-person" trick from Double Q-learning. $Q_{\mathbf{w}}$ picks the action, $Q_{\mathbf{w}^-}$ evaluates it.
2. **Prioritized Replay (Study Smart):** Don't review memories randomly. Focus on transitions where you made the biggest errors (δ). **Analogy:** Focus on your hardest exam mistakes, not the easy questions you already know.
3. **Dueling DQN (Smart Brain Structure):** Splits Q into $V(s) + A(s, a)$. Sometimes the state s is so safe that your action a doesn't matter (e.g., cruising on an empty highway).

Rainbow: Combines all of the above (and more) into one SOTA monster for Atari.

Limitation of Value-Based Methods

- **The "Infinite Choice" Problem:** Hard to use in **continuous** spaces (e.g., drone control). You can't run a max over an infinite number of actions in every step!
- **The "Instability" Trap:** May **diverge** with function approximation. Training a neural network to guess numbers (Q-values) can lead to feedback loops where the values explode to infinity.
- **The "Goal Gap":** Maximizing Q is an *indirect* proxy. You are optimizing a "score guess," which isn't always the same as optimizing the actual **total return** directly.

$\Rightarrow$ Motivates **policy-based** methods:

"Don't guess the score; just learn how to move."

Part 3: Policy-Based RL

Directly optimize the policy

Why Learn the Policy Directly?

Policy-based approach:

- **Direct Mapping:** Input the **state**, and the neural network outputs the **action** (or a probability distribution). No more searching for a max!
- **Continuous Power:** For drone control, it outputs a Gaussian distribution: $\mathcal{N}(\mu_\theta(s), \sigma_\theta(s))$.
- **How it works:** Just sample an action from this distribution. It's natural, smooth, and handles infinite choices easily.

Advantages:

+ Natural for continuous actions (Perfect for physical systems)
+ Stochastic Thinking: Can learn to be unpredictable (e.g., Rock-Paper-Scissors).
− **High Variance:** Learning is noisy. It's like trying to find the peak of a mountain in a foggy storm.

The Policy Gradient Theorem

The Core Intuition: If an action leads to a good result, make it more likely. If it leads to a bad result, make it less likely.

Policy Gradient Theorem (The "Update Rule")

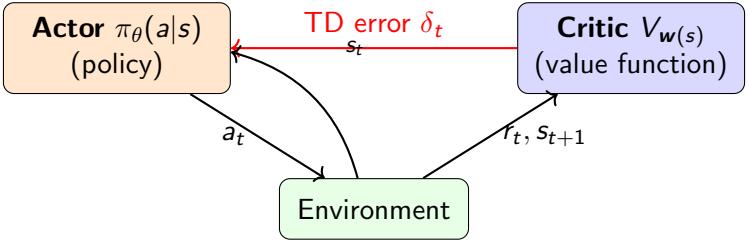
$$\nabla_\theta J(\pi_\theta) \propto \mathbb{E}\left[\underbrace{\nabla_\theta \log \pi_\theta(a|s)}_{\text{Direction to increase prob.}} \cdot \underbrace{Q^{\pi_\theta}(s,a)}_{\text{How "good" it was}}\right]$$

How it works:

- **Good Action ($Q > 0$):** Move θ in the direction that increases its probability.
- **Bad Action ($Q < 0$):** Move θ in the opposite direction to suppress it.
- **Baseline $b(s)$:** Subtract the "average" performance. We only care if an action was *better or worse than average*.

Actor-Critic Methods: Comedian & Audience

The Analogy: Comedian (Actor) tests jokes. Audience (Critic) provides feedback.
The Problem: The old way was like trying to be a stand-up comedian. It was slow and unhelpful. You didn't know which jokes worked.
The Solution (Actor-Critic): Get instant feedback! They learn from the audience's sudden laughs or boos *after every single joke*.



TD error (δ_t): This is the **surprise score**. It's the audience's sudden, loud laugh. It tells the Actor, "This joke landed way better than expected!".

Why Actor-Critic? (The Best of Both Worlds)

Why is this the dominant architecture in Modern RL?

- **1. The Ultimate Combo:** It takes the continuous-action power of Policy methods (the Actor) and pairs it with the step-by-step efficiency of Value methods (the Critic).
- **2. Lower Variance (Less Noise):** Instead of waiting until the end of a long episode to see if you won or lost, the Critic gives **immediate, precise feedback** (Advantage) after every single step.
- **3. Parallel Learning (A2C/A3C):** You can easily clone the Actor! Send 10 parallel agents (or drones) into the environment to explore at the same time and share their knowledge.

Proximal Policy Optimization (PPO)

PPO (Schulman et al., 2017):
Replace KL constraint with a **clipped objective**:

PPO Clipped Objective

$$J^{\text{PPO}}(\pi') = \mathbb{E}\left[\min\left(\frac{\pi'(a|s)}{\pi(a|s)}A^{\pi},\ \ \text{clip}\left(\frac{\pi'(a|s)}{\pi(a|s)},\ 1-\epsilon,\ 1+\epsilon\right)A^{\pi}\right)\right]$$

How it works: (Driving a Car)

- If $A^{\pi} > 0$ (good action): increase $\pi'(a|s)$, but cap at $(1+\epsilon)\times$ old ratio (small update)
- If $A^{\pi} < 0$ (bad action): decrease $\pi'(a|s)$, but cap at $(1-\epsilon)\times$ old ratio
- Prevents too-large policy updates without explicit KL computation

PPO is the most widely used on-policy method (OpenAI default, InstructGPT RLHF, robotics, games).

Deterministic Policy Gradient (DDPG)

Setting: Continuous actions, deterministic policy $a_t = \mu_\theta(s_t)$.

Deterministic Policy Gradient (Silver et al., 2014)

$$\nabla_\theta J \approx \mathbb{E}\left[\nabla_\theta \mu_\theta(s)\, \nabla_a Q_{\boldsymbol{w}(s,a)}\Big|_{a=\mu_\theta(s)}\right]$$

How it works: (Robot Arm — output exact force, not discrete up/down)

- **Actor**: directly outputs a continuous action value
- **Critic**: scores the action with $Q(s, a)$. Actor adjusts to maximize Critic's score
- **Off-policy** with replay buffer + target networks (same tricks as DQN)
- **Exploration**: add random noise to actions (since policy is deterministic)

Pain points:

! Single Critic **overestimates** Q-values (same maximization bias as DQN)

! Actor **exploits** Critic's errors — learns to do things that "look good on paper" but fail in reality

Part 4: Exploration & Key Issues

Practical considerations for RL

Exploration Strategies in RL

ϵ-**greedy:** (Coin Flip)

$$a_t = \begin{cases} \text{argmax}_a\, Q(s_t, a) & \text{with prob } 1-\epsilon \\ \text{random action} & \text{with prob } \epsilon \end{cases}$$

Simple but dumb – explores **uniformly**, wastes time on clearly bad actions.

Boltzmann (softmax) exploration: (Smart Coin Flip)

$$\pi_\tau(a|s) = \frac{\exp(Q(s,a)/\tau)}{\sum_{a'} \exp(Q(s,a')/\tau)}$$

Temperature τ: high $\rightarrow$ uniform (explore), low $\rightarrow$ greedy (exploit).
Assigns **higher probability to more promising actions** – doesn't waste time on obvious losers.

UCB / Thompson sampling: Same ideas from Ch34 bandits, extended to MDPs.

ϵ-Greedy vs. Boltzmann: Comparison (Murphy Table 35.2)

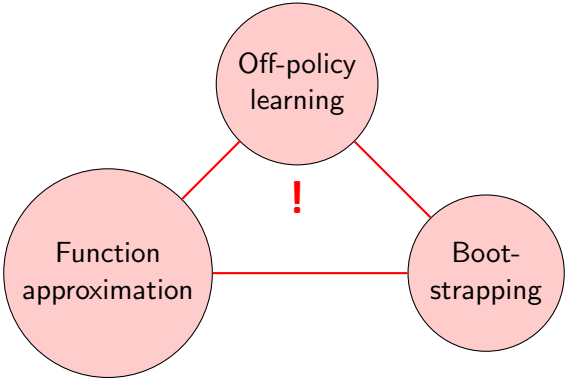
$\hat{R}(s, a_1)$	$\hat{R}(s, a_2)$	$\pi_\epsilon(a_1)$	$\pi_\epsilon(a_2)$	$\pi_\tau(a_1)$	$\pi_\tau(a_2)$
1.00	9.00	0.05	0.95	0.00	1.00
4.00	6.00	0.05	0.95	0.12	0.88
4.90	5.10	0.05	0.95	0.45	0.55
5.05	4.95	0.95	0.05	0.53	0.48
7.00	3.00	0.95	0.05	0.98	0.02
8.00	2.00	0.95	0.05	1.00	0.00

$\epsilon = 0.1$, $\tau = 1$.

Key observation (yellow rows): When Q-values are **close**, ϵ-greedy makes a **sharp jump** ($0.05 \to 0.95$), while Boltzmann transitions **smoothly** ($0.45 \to 0.55$). Boltzmann is more robust to estimation noise.

Off-Policy Learning & The Deadly Triad

Off-policy learning: Learn from data collected by a different policy π_b.
The Deadly Triad (Sutton & Barto): Combining these three can cause **divergence**:



(Chemical Reaction) Each chemical is safe in its bottle. Mix all three $\rightarrow$ explosion. Q-values $\rightarrow \infty$, training completely diverges.

Soft Actor-Critic (SAC) – Haarnoja et al., 2018

Core idea: Don't just maximize reward — also stay **curious**. (The Good Student)

SAC Objective

$$J^{\text{SAC}}(\theta) = \mathbb{E}[R(s,a) + \lambda\,\mathcal{H}(\pi_\theta(s))] \quad = \text{Reward} + \text{Exploration Bonus}$$

How it works:

- Actor outputs a **distribution** (not a single action like DDPG)
- **Twin Critics** take min (prevent overestimation)
- Off-policy with replay buffer + target networks
- λ adjusts automatically — more exploration early, less later

Why SAC wins: Most robust off-policy method for continuous control.
Limitation: Slightly more complex to implement than DDPG/TD3.

As $\lambda \to 0$: entropy vanishes, SAC $\to$ standard RL.

Part 5: Summary & Connections

RL Methods: When to Use What

Scenario	Recommended	Why
Discrete actions	DQN / Rainbow	$\max_a Q$ is easy
Continuous actions (on-policy)	PPO	Stable, simple
Continuous actions (off-policy)	**SAC**	Sample efficient, robust
Very limited data	**Model-based** (MBRL)	Learn from imagination
Known model (games)	MCTS + NN	AlphaZero approach
High-dim observations (images)	Dreamer / DQN	Latent models / CNN
Your UAV project	*SAC + FIS (hybrid)*	*Continuous, partial obs.*

General trend: Off-policy methods (SAC, DDPG/TD3) are preferred for sample efficiency. PPO is preferred when stability matters more than efficiency.

Chapter 35: Summary

1. **RL = learning to act when the model is unknown**
2. **Value-based:** Learn Q^*, derive π^*
   - MC $\rightarrow$ TD $\rightarrow$ Q-learning $\rightarrow$ DQN $\rightarrow$ Rainbow
3. **Policy-based:** Directly optimize π_θ
   - Actor-Critic $\rightarrow$ PPO
   - DDPG for continuous deterministic policies